

Errores

Tema 1. Introducción al análisis numérico

Cálculo Numérico
Diego Rojo



Errores

Errores

- En cálculo numérico, la mayoría de los algoritmos **aproximan** soluciones de problemas matemáticos. La **aproximación** inherentemente introduce **errores**.
- Al realizar los cálculos en un ordenador, una fuente de error inevitable es la **aritmética de precisión finita**.
- En este tema nos centraremos en los errores que surgen de la **representación de números reales** en sistemas informáticos.

Fuentes de error

En cálculo numérico, los errores provienen principalmente de dos fuentes:

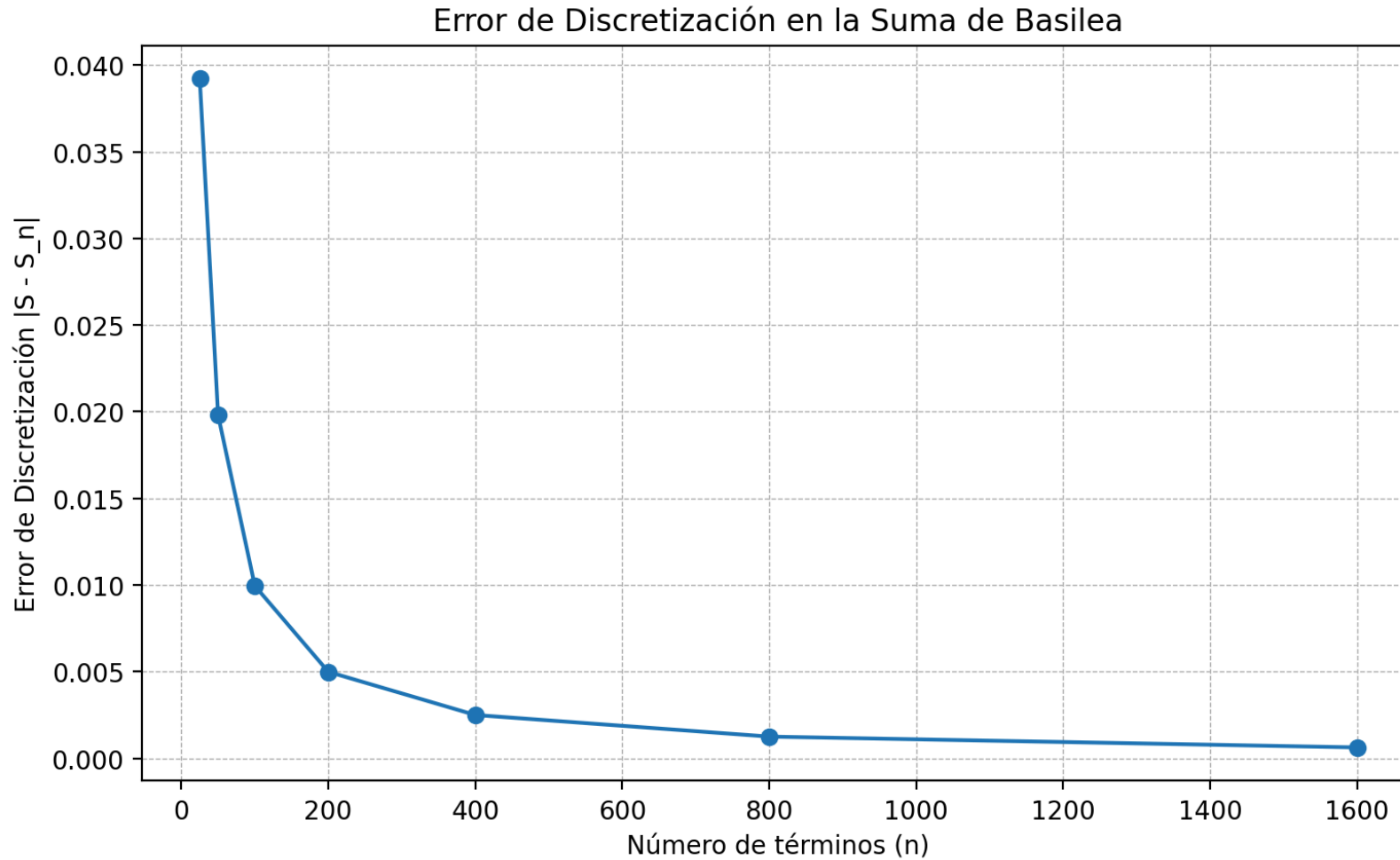
- **Errores de redondeo:** introducidos por las limitaciones de la aritmética del ordenador al representar y operar con números reales.
- **Errores de discretización (o truncamiento):** ocurren al reemplazar un problema matemático continuo (exacto) por una aproximación discreta que el ordenador puede resolver numéricamente.

Errores de discretización

Errores de discretización (o truncamiento)

- Ocurren al **aproximar un problema matemático exacto** (continuo) con un problema **discreto** que el ordenador puede resolver.
- **Ejemplo: Problema de Basilea**
 - Suma infinita: $S = 1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \dots = \sum_{k=1}^{\infty} \frac{1}{k^2}$
 - Solución analítica (Euler): $S = \frac{\pi^2}{6} \approx 1.644934\dots$
 - Aproximación numérica (sumar los primeros n términos) $S_n = \sum_{k=1}^n \frac{1}{k^2}$
 - **Error de discretización:** La diferencia entre S_n y el valor exacto S .

Ejemplo: Problema de Basilea



Errores de redondeo

Aritmética de precisión finita

- Los ordenadores no pueden almacenar números con infinitos dígitos debido a la **memoria finita**.
 - **Ejemplo:** $\pi \approx 3.14 \dots$ (número irracional con infinitos dígitos).
- Existen muchos números reales que **no pueden representarse** de forma exacta en la aritmética del ordenador.
- En los sistemas numéricos computacionales, **no existe continuidad** como en los números reales. Entre dos números de coma flotante consecutivos, existen "**huecos**", es decir, números reales que no son representables.

Ejemplo: $1.1 + 0.1$

- Un ejemplo ilustrativo de la aritmética de precisión finita es la suma `1.1 + 0.1`.
- Matemáticamente, sabemos que el resultado es **exactamente** 1.2.
- Sin embargo, en Python, al evaluar `1.1 + 0.1 == 1.2`, obtenemos `False`.

```
1.1 + 0.1 == 1.2
```

```
False
```

- Entonces, ¿cuál es el resultado de la operación?

Ejemplo: $1.1 + 0.1$ (cont.)

```
print(1.1 + 0.1)
```

```
1.2000000000000002
```

- El resultado mostrado tiene **16 dígitos** decimales. En sistemas estándar con aritmética de doble precisión, solemos confiar en los primeros **15 dígitos** decimales significativos para números reales de orden $O(1)$.
- Una mayor precisión es posible, pero a costa de **más recursos computacionales**.

Error absoluto y error relativo

Notación

- Sea x el **valor exacto** (desconocido en la práctica).
- Sea $\tilde{x}$ un **valor aproximado** (medida, redondeo o resultado algoritmo).
- Definiremos dos formas habituales de cuantificar el error:
 - **Error absoluto**: mide “cuánto” nos desviamos.
 - **Error relativo**: mide “cuánto” nos desviamos **en proporción** al tamaño de x .

Error absoluto

$$\Delta x = \tilde{x} - x$$

Puede ser **positivo o negativo**. Si solo nos importa la magnitud del error:

$$|\Delta x| = |\tilde{x} - x|$$

Cota de error absoluto

A menudo no conocemos Δx , pero sí una **cota**:

$$|\Delta x| \leq \varepsilon \quad \Longleftrightarrow \quad |\tilde{x} - x| \leq \varepsilon$$

Entonces el valor exacto x está en el intervalo:

$$\tilde{x} - \varepsilon \leq x \leq \tilde{x} + \varepsilon$$

Error relativo

Si $x \neq 0$, definimos el **error relativo**:

$$\delta x = \frac{\Delta x}{x} = \frac{\tilde{x} - x}{x}$$

Por tanto:

$$\tilde{x} = x(1 + \delta x)$$

- En la práctica, si la aproximación $\tilde{x}$ es suficientemente precisa, podemos usarla para estimar el error relativo:

$$\delta x \approx \frac{\tilde{x} - x}{\tilde{x}}$$

Relativo vs absoluto

- Relación directa:

$$|\Delta x| = |x| |\delta x|$$

- El error relativo suele expresarse en **porcentaje**:

$$100 \cdot |\delta x| \%$$

- Ideas clave:
 - **Absoluto** es sensible a la **escala** de x .
 - **Relativo** es más comparable entre magnitudes de distinto orden.

Ejemplo

Considera:

- $x_0 = 1.000$, $\tilde{x}_0 = 0.999$
 - $\Delta x_0 = -10^{-3}$, $\delta x_0 = -10^{-3}$
- $x_1 = 10^9$, $\tilde{x}_1 = 10^9 - 1$
 - $\Delta x_1 = -1$, $\delta x_1 = -10^{-9}$
- Aunque $|\Delta x_1| > |\Delta x_0|$, la aproximación de x_1 es muchísimo mejor en términos relativos.

Propagación de errores

Propagación de errores

- Rara vez usamos un único dato: combinamos aproximaciones en operaciones.
- Pregunta típica: **si a y b tienen error, ¿qué error tendrá $x = f(a, b)$?**

Suma y resta

Sea $x = a \pm b$ con aproximaciones $\tilde{a}, \tilde{b}$.

- Resultado clave:

$$|\Delta x| \leq |\Delta a| + |\Delta b|$$

Ejemplo

Si:

$$|4.56 - a| \leq 0.14, \quad |1.23 - b| \leq 0.03$$

entonces para $x = a - b$:

- $\tilde{x} = 4.56 - 1.23 = 3.33$
- Cota:

$$|3.33 - x| \leq 0.14 + 0.03 = 0.17$$

Cancelación catastrófica

Caso delicado: $x = a - b$ con $a \approx b$.

- El denominador $|a - b|$ puede ser pequeño $\Rightarrow$ el **error relativo de x se amplifica**.
- Cota:

$$|\delta x| \leq \frac{|\Delta a| + |\Delta b|}{|a - b|}$$

Esto explica por qué la resta de números “casi iguales” puede ser numéricamente peligrosa.

Ejemplo de cancelación catastrófica

Si:

$$|1.25 - a| \leq 0.03, \quad |1.20 - b| \leq 0.03$$

Entonces $\tilde{x} = 1.25 - 1.20 = 0.05$.

Aunque a y b tienen errores relativos del orden del **2–3%**, la cota del error relativo del resultado puede superar el **100%**.

Multiplicación y división

Sea $x = ab$ (o $x = a/b$).

- Cota (primer orden):

$$|\delta x| \leq |\delta a| + |\delta b|$$

Ejemplo

Con los mismos datos:

$$|1.25 - a| \leq 0.03, \quad |1.20 - b| \leq 0.03$$

Errores relativos aprox.:

$$|\delta a| \approx \frac{0.03}{1.25} \approx 2.4\%, \quad |\delta b| \approx \frac{0.03}{1.20} \approx 2.5\%$$

Entonces:

$$|\delta x| \leq 2.4\% + 2.5\% = 4.9\%$$

Reglas

- **Suma / resta** → sumar **errores absolutos**:

$$|\Delta(a \pm b)| \leq |\Delta a| + |\Delta b|$$

- **Producto / cociente** → sumar **errores relativos**:

$$|\delta(ab)| \leq |\delta a| + |\delta b|, \quad |\delta(a/b)| \leq |\delta a| + |\delta b|$$

- **Cancelación catastrófica**: cuidado con $a - b$ si $a \approx b$.

Ejemplo: $\lim_{x \rightarrow 0} \frac{1 - \cos(x)}{x^2}$

Consideremos la función

$$f(x) = \frac{1 - \cos(x)}{x^2}$$

- $f(x)$ no está definida en $x = 0$.
- ¿Cuál es el límite $\lim_{x \rightarrow 0} f(x)$?

Límite con la regla de L'Hôpital

$$\lim_{x \rightarrow 0} \frac{1 - \cos(x)}{x^2} \stackrel{L'H}{=} \lim_{x \rightarrow 0} \frac{\sin(x)}{2x} \stackrel{L'H}{=} \lim_{x \rightarrow 0} \frac{\cos(x)}{2} = \frac{1}{2}$$

Límite con SymPy

- Podemos usar `SymPy` para calcular el límite simbólicamente:

```
import sympy

x = sympy.symbols('x')
f_sym = (1 - sympy.cos(x)) / x**2
limit_sym = sympy.limit(f_sym, x, 0)
print(limit_sym)
```

- `SymPy` nos proporciona el límite: $\frac{1}{2}$.

Límite mediante aproximación

- Estimación numérica con Python.
- Evaluamos $f(x)$ para valores de $x = x_i = 10^{-i} \approx 0$, con $i = 5, \dots, 11$.
- Teóricamente, los valores de $f(x_i)$ deberían aproximarse a 0.5 a medida que x_i se acerca a 0.

```
import numpy as np

f = lambda x: (1 - np.cos(x))/x**2
xi = (1.e-5, 1.e-6, 1.e-7, 1.e-8, 1.e-9, 1.e-10, 1.e-11)
for x in xi:
    print(f'f({x}) = {f(x)}')
```

Límite mediante aproximación

- Inicialmente, la convergencia parece correcta, pero para valores muy pequeños de x , los resultados se vuelven **incorrectos**, tendiendo a 0 en lugar de 0.5.
- Esto se debe a la **cancelación catastrófica**: para valores muy pequeños de x , la diferencia $1 - \cos(x)$ es un número muy cercano a 0.

Solución: Reformulación

- Para mejorar la estabilidad, podemos **reformular** $f(x)$ para evitar la resta de números casi iguales.
- Realizamos transformaciones algebraicas:

$$f(x) = \frac{1 - \cos(x)}{x^2} = \frac{\sin^2(x)}{x^2(1 + \cos(x))}$$

- Esta nueva fórmula **evita la cancelación catastrófica** para $x \approx 0$.

Falta de Asociatividad

Falta de asociatividad

- La **falta de asociatividad** en la aritmética de coma flotante significa que el orden en que se realizan las operaciones puede afectar el resultado final.
- En la aritmética real, la suma y la multiplicación son asociativas:
 - $(a + b) + c = a + (b + c)$
 - $(a \times b) \times c = a \times (b \times c)$
- Sin embargo, esto **no siempre** se cumple en la aritmética de coma flotante debido a la precisión finita (errores de redondeo).

Ejemplo de falta de asociatividad

- Consideremos la suma de tres números con magnitudes muy diferentes:
 - $x = 1.0 \times 10^{20}$
 - $y = -1.0 \times 10^{20}$
 - $z = 1.0$
- En la aritmética "verdadera", $(x + y) + z = x + (y + z) = 1.0$.
- Veamos qué ocurre en la aritmética de coma flotante.

```
x = 1.0e20
y = -1.0e20
z = 1.0

print(f'(x + y) + z = {(x + y) + z}')
print(f'x + (y + z) = {x + (y + z)}')
```

Precisión de la Máquina

Precisión de la máquina

- Otro fenómeno inesperado en la aritmética computacional es la **no unicidad del cero** al sumar con uno.
- Sabemos que en matemáticas $0 + 1 = 1$.
- Sin embargo, en un ordenador, existen números **distintos de cero** que, al sumarse a 1, dan como resultado 1 debido a la precisión finita.
- El **épsilon de la máquina**, denotado EPS, es el número positivo más pequeño tal que $1 + \text{EPS} \neq 1$.
- Para cualquier número x entre 0 y EPS (excluyendo 0 y EPS), $(0 < x < \text{EPS})$, se cumple que $1 + x = 1$ en la aritmética del ordenador.

Cálculo del épsilon de la máquina

Podemos aproximar el épsilon de la máquina mediante el siguiente algoritmo iterativo:

1. Inicializar $x = 1.0$.
2. **Mientras** $1.0 + x \neq 1.0$:
 - Dividir $x = x/2.0$.
3. Definir $\text{EPS} = 2.0 \cdot x$.

Implementación en Python

```
x = 1.0
while (1.0 + x != 1.0):
    x = x / 2.0
EPS = 2.0 * x
print(f'EPS = {EPS}')
print(f'x = {x}')
```

- Este código localiza el valor de `x` tal que la suma `1.0 + x` se redondea a `1.0`.
- Multiplicamos por 2 para obtener una aproximación del épsilon de la máquina, EPS.